Architettura dello Scaffolding Context-Aware per AI

Guida operativa per Claude e Google Antigravity / Gemini: regole, progressive disclosure, automazione e sinergie.

---

1. Come Funziona il Workflow di Sessione (Zero Token Waste)

Caricare documentazione intera distrugge il budget di contesto e genera allucinazioni. L'architettura adottata utilizza una struttura gerarchica a **recupero chirurgico on-demand**:

- **Root Guard (CLAUDE.md e GEMINI.md):** Caricati come regole di root all'avvio (< 60 righe). Contengono invarianti globali, regole di branching/licenza e la tabella di consultazione.
- **Index Map (docs/progress.md):** L'unico file indice. Non viene letto per intero ogni volta: serve solo per instradare verso la scheda d'area corretta.
- **Estrazione Mirata via Grep/Offset:** L'AI consulta direttamente i task (grep -n -A 15 '^## Open work' docs/progress/<area>.md) o le regole (grep -n -A 20 '^## Rules' ...) consumando ~100 token invece di 10.000.

Fase	Azione dell'AI	File Coinvolti & Output
1. Boot	Caricamento regole base e comandi rapidi.	CLAUDE.md / GEMINI.md (~50 righe)
2. Routing	Identificazione dell'area target se non ovvia.	docs/progress.md (solo lookup)
3. Lookup	Estrazione chirurgica di task aperti e vincoli.	grep -n -A 15 '^## Open' docs/progress/<area>.md
4. Esecuzione	Ispezione mirata (AST/cindex) e test selettivi.	Sorgenti puntuali, make test-<area>
5. Chiusura	Protocollo Leave-No-Trace: update stato e audit.	docs/progress/<area>.md, docs_audit.py

2. Struttura dei File da Creare in un Nuovo Progetto

Per evitare il degrado nel tempo, la documentazione deve essere ripartita in 3 livelli a ciclo di vita separato:

1. **File Operativo Globale (CLAUDE.md e GEMINI.md):** Massimo 60 righe. Solo comandi, test, guardrail e indice scorciatoie. Zero cronologia o log di release.
2. **Living Progress (docs/progress/<area>.md):** Schede di lavoro attive da 80–120 righe. Contengono solo lo stato attuale del codice, task aperti (## Open work) e regole fisse (## Rules).
3. **Design & RFC (docs/design/<topic>.md):** Idee, proposte evolutive, architettura a lungo termine. Essendo testi lunghi e speculativi, non devono stare in progress per non inquinare l'audit dei token.

3. Focus Google Antigravity & Gemini: Differenze e Punti di Forza

Mentre Claude si affida a CLAUDE.md, Google Antigravity adotta un'architettura enterprise modulare:

- **File Primario GEMINI.md (o AGENTS.md):** Ha precedenza assoluta. Dispone di un budget di regole dedicato (20.000 token separato dal contesto generale) con troncamento intelligente e switch a puntatori on-demand se si supera il limite.
- **Progressive Disclosure Nativa:** Le skill in .agents/skills/ o .claude/skills/ espongono solo metadati brevi (name e description) nel prompt; il corpo esteso viene caricato solo se il modello decide di attivarle.
- **Mobile / Cloud Relay tramite GitHub Actions:** Grazie a .github/workflows/antigravity-agent.yml, l'agente può essere lanciato dallo smartphone tramite i runner di GitHub Actions senza consumare risorse locali.
- **Subagent Paralleli Asincroni:** Antigravity supporta nativamente subagent con tool differenziati (es. research in sola lettura per non inquinare il contesto, self per esecuzioni indipendenti).

4. Cosa Claude Può Ereditare da Antigravity / Gemini

Possiamo arricchire l'ambiente Claude integrando i seguenti pattern mutuati da Antigravity:

1. **Subagent 'Research' Separato per Ricerche Pesanti:** Invece di far leggere decine di file a Claude nella sessione principale, delegare a un subagent rapido (es. Haiku/Sonnet in sola lettura) che restituisce solo un riassunto finale da 5 righe.
2. **Progressive Disclosure per le Istruzioni:** Spostare procedure complesse in skill on-demand con YAML frontmatter, mantenendo CLAUDE.md asciutto al minimo indispensabile.
3. **Deduplicazione Cross-Agent:** Mantenere CLAUDE.md e GEMINI.md perfettamente allineati e sincronizzati sullo stesso indice (docs/progress.md), così che qualsiasi membro del team possa usare l'uno o l'altro assistente con le medesime garanzie.

5. Template Universale di Avvio: CLAUDE.md & GEMINI.md

Utilizza questa struttura alla radice del repository per garantire compatibilità con entrambi gli agenti:

```
# [Nome Progetto] — Note Operative di Sessione (Claude & Gemini)
Repository [org/repo], branch `main`. Stack: [linguaggi, architettura, OS].

## 1. Recupero Mirato (Zero Spreco Token)
Non leggere file per intero. Lo stato attuale e l'indice sono in `docs/progress.md`.
- Trovare cosa c'è da fare: `grep -n -A 15 '^## Open work' docs/progress/.md`
- Verificare vincoli/regole: `grep -n -A 20 '^## Rules' docs/progress/.md`
- Ricerca definizioni/funzioni: `python scripts/cindex.py  ` o `grep -n`

## 2. Regole di Ingaggio
- Branching: Creare branch `feat/` prima di sviluppare. Mai lavorare su `main`.
- Test Selettivi: Lanciare solo i test d'area toccati (es. `make test-`).
- Nessun segreto (chiavi API, token) committato nei sorgenti.

## 3. Subagent Specializzati
- `research` (o `light`): Esplorazione read-only e ricerca documentazione.
- `standard` / `self`: Implementazione feature e test su più file.
- `tracker`: Aggiornamento spunte e task in `docs/progress/`.

## 4. Chiusura Task & Protocollo Leave-No-Trace
Prima di terminare o dare la risposta finale:
1. Aggiorna `## Open work` nella scheda d'area (rimuovi o spunta compiti finiti).
2. Se hai introdotto un vincolo o pattern, aggiungilo in `## Rules (do not break)`.
3. Esegui `python scripts/docs_audit.py score` per validare link e assenza di orfani.
```

6. I Tre Pilastri dell'Automazione Continua

A. Git Pre-Commit Hook (.git/hooks/pre-commit): Esegue python scripts/docs_audit.py score e blocca il commit locale se ci sono link rotti o file orfani.

B. Target Makefile / Build Tool: Inserire il target test-docs nella suite dei test standard (make test o make test-host).

C. CI/CD GitHub Actions: Uno step dedicato Check documentation audit (python3 scripts/docs_audit.py score) all'interno del workflow di build/test su PR.